

SP106 - Electricity Load Forecasting with LSTMs

FINAL REPORT DOCUMENT

Macsen Higgins & Jacob Southern

CS 4850 - Section 02

Spring 2026

Professor Sharon Perry

May 4, 2026

Visit our website here: <https://macsenhigginsksu.github.io/CS-4850-SP106/>

View our repository here: <https://github.com/MacsenHigginsKSU/CS-4850-SP106>

STATS AND STATUS		
Lines of Code (LOC)	650 (Python), 500 (HTML), 50 (JS)	
Components/Tools	Original Dataset, Python Code, Predictions Dataset, HTML/JS Frontend	
Hours Estimate	100	
Hours Actual	85	
Status	Project is complete	

Table of Contents

1.0 Introduction	3
1.1 Abstract.....	3
1.2 SDLC	4
2.0 Design Constraints	4
2.1 Environment	4
2.2 User Characteristics	4
2.3 System.....	5
3.0 Requirements.....	5
3.1 Functional Requirements.....	5
3.2 Nonfunctional Requirements	5
4.0 System Design	6
4.1 System Architecture	6
5.0 Development	6
5.1 Implementation Details	6
5.2 Inter-Component Connections	7
5.3 Environment Setup and Configuration	8
6.0 Frontend Test Plan	8
6.1 Purpose	8
6.2 Scope.....	8
6.3 Release Content	9
6.4 Regression Testing	9
6.3 Platform Testing.....	9
7.0 Backend Test Plan	9
7.1 Test Level Responsibility	9
7.2 Test Type and Approach	9
7.3 Build strategy	10
7.4 Test Execution Schedule	11
7.4.1 Test environment.....	11
7.4.2 Access to other applications.....	11
7.4.3 Testing Requirements.....	11
7.4.4 Data Requirements	12
7.4.5 Resources & Skills	12

7.5 Testing Tools	12
7.6 Testing Handover Procedure	12
7.7 Testing Metrics	13
7.8 Test Environment Plan	13
7.9 Test Environment Details.....	13
7.9.1 Testers.....	13
7.9.2 Hardware and Firmware	13
7.9.3 Software.....	13
7.9.4 Interfaces	13
7.9.5 Other Materials.....	13
7.10 Environment Control	14
7.11 Environment Roles and Responsibilities.....	14
7.12 Administrative Plan.....	14
7.12.1 Approvals	14
7.12.2 Training	14
7.12.3 Defect Management.....	14
8.0 Version Control	14
8.1 GitHub Tree Visualizer	14
8.2 Mermaid Flowchart	15
8.3 GitHub Commit History.....	16
9.0 Conclusion.....	17
10.0 Appendix	17
10.1 Data Sources	17
10.2 Definitions.....	17
10.3 Points of Contact	18

1.0 Introduction

1.1 Abstract

Given a dataset from PJM Interconnection LLC documenting system-wide energy consumption along the Eastern United States from 1998 to 2018, we aim to develop a forecasting model using Long Short-Term Memory (LSTM) modeling that

can accurately predict usage based only on date and time. Development will operate using TensorFlow in Python for LSTM modeling. Users will be able to input a value for date-time and receive predicted energy usage estimate with descriptive statistics. TensorFlow is an open-source ecosystem for machine learning available for multiple coding languages including Python.

1.2 SDLC

Phase I: Planning and Requirements –

Strictly defining the scope through functional and non-functional requirements.

Phase II: Data Analysis & Design -

Exploratory Data Analysis (EDA) Notebook: A Jupyter Notebook showing visualizations of the PJM dataset. It will have histograms of energy usage, linecharts showing seasonality, and heatmaps of missing values.

Preprocessing script: A script that can clean and parse the data for the LSTM to read.

Model Architecture Diagram: A visual diagram showing the input layer -> LSTM layers -> dense output layer

Phase III: Implementation -

Phase IV: Testing and Validation -

Model Evaluation Report: A section within the final report that details performance metrics. Thinking about using Mean Absolute Error to compare against a baseline.

Prediction vs. Actual: The visualization portion of the predictions. Visually demonstrate accuracy.

2.0 Design Constraints

2.1 Environment

Computing Resource Constraints -

The model training, testing and inference programs must run on a standard consumer-grade CPU

Software Dependencies -

The system must utilize Python 3 and TensorFlow without the use of other dependency-heavy tools

2.2 User Characteristics

Technical Proficiency -

The user is assumed to be comfortable with basic statistical concepts, but otherwise experience relating to the topic of the study is not required.

2.3 System

Algorithm Limitations -

The program must use LSTM networks trained over many generations to approximate total energy usage in MW based on date and time alone.

Input Dimensionality -

Predictions are made only based on date and time (excluding weather data).

3.0 Requirements

3.1 Functional Requirements

FR-01: The system will accept the dataset (CSV format) covering the years 1998-2018 and parse the datetime and energy_consumption columns without errors.

FR-02: The user interface will allow the user to input a specific target date and time for prediction using a scrolling date and time picker.

FR-03: The system will load the pre-trained LSTM TensorFlow model and generate a numerical energy consumption prediction based on the user's input.

FR-04: In addition to the prediction, the system must calculate and display descriptive statistics for that specific time window.

FR-05: The hosting site for the model will have a separate section that demonstrates testing results based on the entire dataset with a written description alongside it.

3.2 Nonfunctional Requirements

NFR-01: The LSTM model must display and achieve a Mean Absolute Error lower than our chosen threshold of 'X' (X will be properly defined once data has been collected).

NFR-02: The system will generate the prediction within X seconds of the user submitting the datetime query.

NFR-03: The data splitting process will be reproducible using a fixed random seed in the code, ensuring the test set is not seen during training

NFR-04: The application must run successfully in a standard python virtual environment using TensorFlow, NumPy, and pandas

NFR-05: The date picker and predictor tool will be intuitive and easy to understand to the average user, with results being simple to interpret.

4.0 System Design

4.1 System Architecture

At its core, our model will use data-pipeline architecture. There are three different subsystems of the architecture: an offline portion where the model is trained, an online portion where the user asks for a prediction, and an in-between layer where important data is stored.

The offline portion is the subsystem that runs on the developer machine. It imports the raw PJM Electricity Usage dataset (.csv) and reads each value using a python library (pandas). Our parser will parse and handle the data and convert each time value into cyclical math. It will also normalize the megawatts into a range of $[0,1]$. Once all the data is cleaned and processed, it is fed into the TensorFlow LSTM network.

The online portion's job is to display a user interface with simple instructions. The user will input a date-time value using Python web app Streamlit, pass through the LSTM prediction model, and get converted back into megawatts. At the moment, there are no further features planned for the UI.

Lastly, and most importantly, the bridge layer will connect the offline and online portions. Its responsibility is to convert the user input into the LSTM format, then denormalize and contextualize the output.

The architecture will be hosted and organized on GitHub. The repository will once again be split into its online and offline portions. The offline portion will contain the Jupyter notebooks, python scripts, and predictive model. The online portion (using github.io) will host the website and all user interface aspects.

5.0 Development

5.1 Implementation Details

Component 1: Preprocessing Block

Our preprocessing implementation consists of three steps. First is direct data ingestion. The eleven spreadsheets of data provided by Kaggle are split up, parsed, and cleaned of erroneous data. Next is scaling the features, making sure that the elements are arranged in a cyclical fashion in regard to time. Last is transforming the data into piecewise functions called splines that properly model non-linear patterns and remove extraneous noise from the data.

At the end of preprocessing, each individual point has a given date-time format, and the entire dataset is split in an 80:20 ratio for training and testing respectively.

Component 2: LSTM Model Training

Our model architecture utilizes Long Short-Term Memory (LSTM) layers to capture the temporal dependencies inherent in the time-series data. This stage involves feeding the preprocessed splines into the network, where the model learns to identify long-term trends and short-term fluctuations. We implement dropout layers to prevent overfitting. The training process is iterative, continuing until the loss function stabilizes, ensuring the model can generalize effectively from the training subset to unseen data.

Component 3: Prediction Results and Interpretation

Once trained, the model generates forecasts by projecting the learned patterns into the testing window. By comparing the predicted splines against the actual 20% test holdout, we can interpret the model's reliability and identify specific conditions where the non-linear patterns diverge from reality.

Component 4: Website UI

The final component serves as the user interface, translating our backend computations into an accessible web-based dashboard. Built with a clean, intuitive interface, the site allows users to input a specific date and time via a date picker and dropdown menu. Upon triggering the "Run Forecast" action, the backend passes the input to LSTM pipeline to generate an instantaneous prediction. The result is then displayed prominently in Megawatt-hours (MWh).

Using Streamlit, we visualize the results through interactive charts that allow users to toggle between raw data and LSTM predictions. This display provides a clear, visual confirmation of the project's efficacy, offering stakeholders a streamlined view of the data trends, model performance metrics, and final forecasted outcomes.

5.2 Inter-Component Connections

Our project does not use any external databases, so most of this section is not applicable. Our data comes from a dataset on Kaggle called Hourly Energy Consumption published by data scientist Rob Mulla, which contains electricity grid load data for regional transmission organization PJM Interconnection spanning from 1998 to 2018. The files contained in this dataset are small enough to be stored and operated upon locally, mitigating the need for any cloud computing or cloud storage integration in our project.

There are no complex configuration details for our data connections since they are stored locally. The CSV files containing our source data is stored in a subfolder of our main project folder and we access them directly in our Python code using this local environment.

Our code communicates with our dataset using the `read_csv()` method in Python's pandas package, which converts the values into a dataset that we can easily operate on. Although fourteen different data files are available as a part of the

Kaggle dataset, we decided to settle on `pjm_hourly_est.csv` as our data source to build this project from, since it contains the electricity load records from all twelve different providers across a twenty-year span, whereas most other files are isolated to a single provider. Initial iterations of our code included an R file dedicated to cleaning and mutating the data from the CSV file from Kaggle, but all data processing is now handled in Python alongside the rest of our code.

5.3 Environment Setup and Configuration

Development was handled in Microsoft VS Code. Each developer was responsible for setting up a virtual environment for Python 3.13.12 on their system, and Python execution was handled within a master Jupyter notebook. Live web demo add-ons were utilized for frontend development.

6.0 Frontend Test Plan

6.1 Purpose

The purpose of this document is to define how our project can fit into the standards of a software testing plan, given its unorthodox structure and limited user interactivity. The most important aspect of our project is preprocessing existing data and structuring our LSTM model such that its training results in a highly accurate predictor of energy usage based on date and time. Since most of this work is done behind the scenes before the possibility of user input comes into play, our test plan will mostly center around preventing faults in the web UI we are developing to showcase our model. We will ensure compatibility across multiple device types and prevent any faulty communication between the user and the model as stored in JavaScript.

6.2 Scope

As previously stated, the scope of our testing will be limited to the front-end companion website that users will be able to interact with and read analytical results about the model's performance. There are many facets of the training process in Python where we will test the model's performance to evaluate whether changes need to be made, but those tests are more closely related to trial-and-error research than software testing in the engineering sense, which influenced our decision to restrict our scope in such a manner.

In scope -

- Unit tests on frontend UI elements
- Integration tests between frontend UI and trained model as stored in JavaScript

- Functional testing on outputs of model predictions (does the output align with what you might expect?)
- Performance testing on processing time for predictions

Out of scope -

- Backend testing
- Model development testing

6.3 Release Content

The only content from our project that we are releasing to the public is our website that will allow users to see usage predictions from their choice of date and time, as well as read published insights from the development of the model.

6.4 Regression Testing

Since our UI will not receive continuous updates, regression testing is not necessary.

6.3 Platform Testing

We will test our site's functionality on desktop and mobile devices to ensure compatibility across multiple platforms.

7.0 Backend Test Plan

7.1 Test Level Responsibility

Test Level	External Party	Project Team	Business
Unit Testing		P	
Integration Testing		P	
Security Testing	P	S	
Connectivity Testing		P	
User Acceptance Testing		S	P
Production Verification Testing		S	P

7.2 Test Type and Approach

Test Type	Objectives
Unit Testing	To verify that individual code components, such as the Min-Max scaling functions, the LSTM gate logic,

	and the time-series windowing scripts, function correctly in isolation.
Integration Testing	To ensure the seamless flow of data between the Python-based preprocessing pipeline, the TensorFlow/Keras model architecture, and the final output visualization layer.
Security Testing	To safeguard the model's integrity by checking for vulnerabilities in the web interface and ensuring that API endpoints for data ingestion are protected against unauthorized access or injection.
Connectivity Testing	To validate the stable connection between the GitHub Pages hosting environment and any external data sources or storage used to retrieve historical energy consumption records.
User Acceptance Testing (UAT)	To confirm with stakeholders that the model's forecasting accuracy and the web dashboard's usability meet the business requirements for monitoring grid stress and energy demand.
Production Verification Testing (PVT)	To conduct a final "smoke test" in the live environment (GitHub Pages) after deployment to ensure the model loads correctly and generates forecasts using real-time data inputs without errors.
Stress & Volume Testing	To evaluate the system's performance when processing 20+ years of hourly energy data simultaneously and to verify the application remains responsive under high concurrent user traffic.
Disaster Recovery Testing	To verify that model weights and preprocessing configurations are backed up and can be fully restored within the defined Recovery Time Objective (RTO) in the event of a site or host failure.

7.3 Build strategy

The build strategy for the LSTM project follows a phased modular implementation approach:

Step 1: Data engineering build -

Deployment of the PJM Interconnection dataset ingestion and preprocessing pipelines. This involves the normalization, windowing, and spline initialization of the data.

Step 2: Model core build -

Implementation of the LSTM neural network architecture and training logic.

Step 3: Interface and deployment build -

Integration of the predictive engine with the GitHub Pages web interface for visualization.

7.4 Test Execution Schedule

Testing Phase	Function	Priority	Duration
Unit Testing	Validates individual gate logic and scaling functions.	High	Week 1
Integration Testing	Verifies data flow from Python scripts to TensorFlow.	High	Week 1-2
Connectivity Testing	Ensures secure handshake with GitHub Pages/Data APIs.	Medium	Week 2
S&V Testing	Tests 20 years of hourly data processing under load.	Medium	Week 3
UAT	End-user verification of forecasting accuracy/UI.	High	Week 3-4
PVT	Final smoke test on the live production URL.	High	Post-Launch

7.4.1 Test environment

- Local Dev Environment: High compute workstations with high GPU support for model training.
- Staging environment: A private GitHub repository branch and a local host server to simulate the web interface
- Production: GitHub Pages

7.4.2 Access to other applications

- GitHub/GitLab: for version control and deployment hosting
- PJM Data Portal/API: For sourcing the 20 years of energy consumption records
- TensorFlow/Keras Libraries: For the deep learning execution environment.

7.4.3 Testing Requirements

- Local Python Runtime: Access to an environment with Python 3.x, TensorFlow, and Pandas installed.
- Web Browser: Access to Chrome/Edge for testing the GitHub Pages interface.

- Repository Access: "Developer" or "Tester" permissions for the project's GitHub repository.
- Defect Tracker: Access to the Project Issue Log (Excel or GitHub Issues) for reporting bugs.

7.4.4 Data Requirements

Testing cannot start until the following data setup is complete:

- Baseline Dataset: The PJM Interconnection hourly load data must be cleaned and stored in a reachable database format.
- Test Data: The dataset is split into two groups, a tester group and model group. It is created to test the error handling and model resilience.
- Normalized Scaffolding: Precalculated mean/standard deviation values for inverse scaling validation.

7.4.5 Resources & Skills

- Data Scientist/ML Engineer: Expertise in Python and LSTM architectures to debug gradient issues or accuracy drops.
- Data Analyst: Proficiency in SQL/Pandas to verify data integrity and historical consistency.
- Web Developer: Understanding of JavaScript/HTML for troubleshooting the GitHub Pages deployment.
- QA Specialist: General understanding of Regression Testing and time-series validation.

7.5 Testing Tools

Process	Tool
Test Case Creation	Microsoft Word / GitHub Markdown
Test Case Tracking	Microsoft Excel
Test Execution (Functional)	Manual & Automated (PyTest / TensorFlow)
Test Execution (Performance)	Apache JMeter / Locust
Test Management	Microsoft Excel
Defect Management	GitHub Issues / Microsoft Excel
Code Coverage	Coverage.py

7.6 Testing Handover Procedure

Before the LSTM model is accepted for system-level testing, the development team must provide a report showing at least 80% code coverage and zero critical or high defects. Then, the testing team will demonstrate the model's performance on the PJM test set to any stakeholders. Handover is contingent on the completion of all integration and stress tests.

Any unresolved low/medium priority defects will be documented in the defect log with a signed-off mitigation plan before moving on to the next environment. The test completion memo must be signed by the project lead to formally transition the model from the staging branch to the production GitHub Pages site.

7.7 Testing Metrics

Testing metrics for the LSTM project will focus on Defect Density, Forecast Accuracy (MAPE/RMSE), and Test Execution Rates. By tracking the volume of bugs found within specific modules—such as the data preprocessing pipeline versus the model layers—the team can identify stability issues and prioritize rework.

These metrics will be captured in the Excel tracking tool to provide objective feedback on model performance and project progress, ensuring the final energy forecasts are reliable before the production launch.

7.8 Test Environment Plan

The environment follows a tiered progression from local development to cloud-based visualization.

7.9 Test Environment Details

7.9.1 Testers

2 Testers involved, one as the lead machine learning engineer, and the other as the data analyst and quality assurance.

They must have administrative access to the GitHub repository, with read/write permissions and access to the local GPU.

7.9.2 Hardware and Firmware

Unnecessary for the scope of our project.

7.9.3 Software

Software Name	Version	Type	Provider
Windows 11 / Linux	Latest	Operating System	Internal
Python	3.10+	Programming Language	Open Source
TensorFlow / Keras	2.15+	Deep Learning Framework	Open Source
Pandas / NumPy	Latest	Data Manipulation	Open Source
GitHub Desktop	Latest	Version Control Tool	GitHub

7.9.4 Interfaces

The primary interface is the PJM Data API/Filestream which feeds historical energy metrics into the LSTM preprocessing module.

7.9.5 Other Materials

The lead dev will provide a technical manual, primarily detailing the LSTM gate logic and scaling parameters.

7.10 Environment Control

So long as the environment is set up to a state of functionality, no further control is required.

7.11 Environment Roles and Responsibilities

Role	Staff Member	Responsibilities
Release Manager	Macsen Higgins	Responsible for overall establishment, coordination and support of the test environment.
Test Manager	Jacob Southern	Responsible for advising release manager of environment requirements for planning, establishment and ongoing support.
Project Manager	Macsen Higgins	Escalation point for environment issues.

7.12 Administrative Plan

7.12.1 Approvals

As a team of two people, a hierarchy for approvals is not necessary for our tests, as we believe we can share the responsibility of auditing testing criteria.

7.12.2 Training

Training will not be required for this testing due to its limited scope.

7.12.3 Defect Management

Detail how defects will be managed for this project. Detail what defect management tool will be used. Reference any defect process if required.

8.0 Version Control

Throughout the course of this project, we have used GitHub for version control and collaborative development. The full repository can be viewed at <https://github.com/MacsenHigginsKSU/CS-4850-SP106>.

8.1 GitHub Tree Visualizer

Here is the organization of our GitHub and some descriptions of each filetype.

CS-4850-SP106/

├── .gitignore	<i>file in a Git repository that instructs Git which files to ignore</i>
├── README.md	<i>contains relevant background and instructions for the project</i>
├── TensorFlow.ipynb	<i>where the data was cleaned and the model was trained</i>
├── assets/	<i>folder containing assets for the website deployment</i>
├── images/	
├── KSU-Logo.jpg	
├── KSU_ccse_logo.png	
├── ksu-logo2.png	
└── sp_jacob.png	

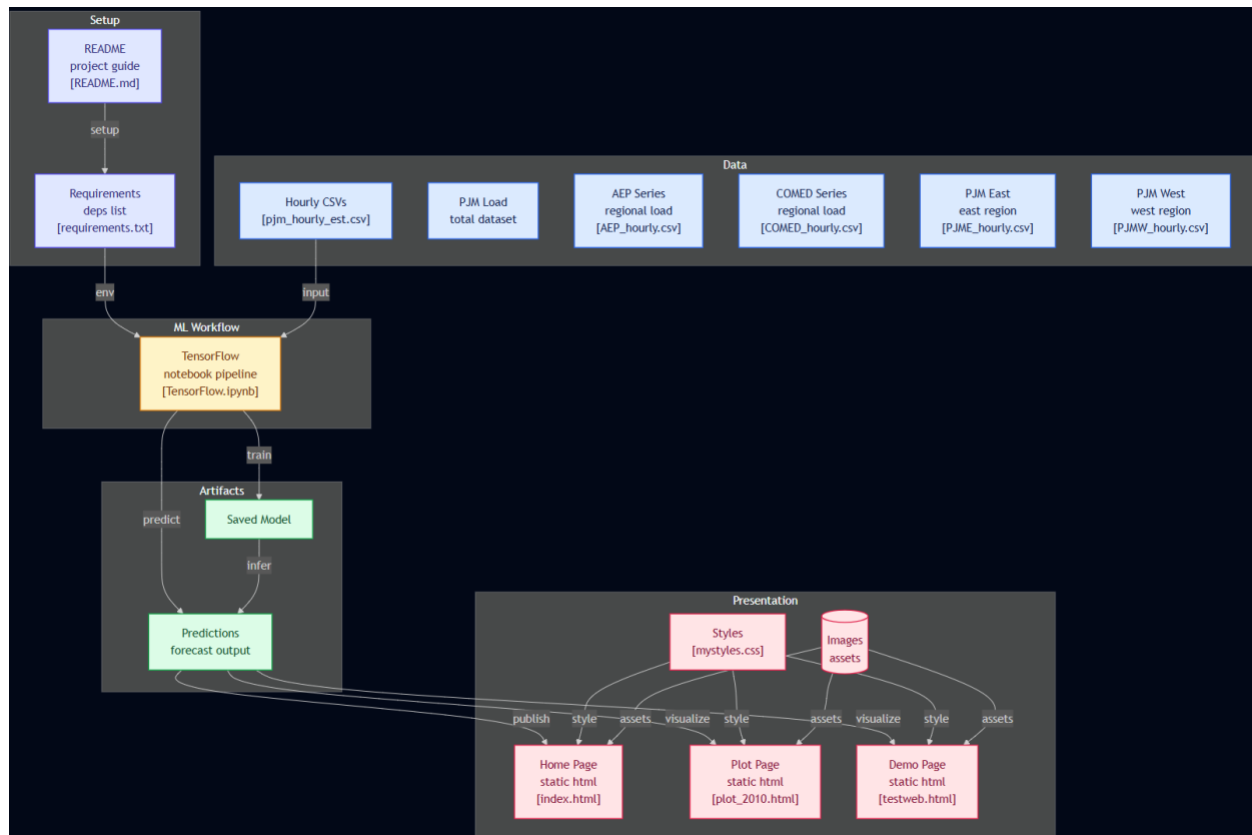
```

| | └─ sp_macsen.png
| | └─ macsen_resume.pdf
| | └─ mystyles.css      | css file to maintain a consistent style of our website
| | └─ plot_2010.html   | using plotly, embedded an interactive graph to visualize the data
| └─ demo.html          | html code for the demo page
| └─ electricity-data/  | our data can be sourced from this folder, sourced via Kaggle
|   └─ AEP_hourly.csv
|   └─ COMED_hourly.csv
|   └─ DAYTON_hourly.csv
|   └─ DEOK_hourly.csv
|   └─ DOM_hourly.csv
|   └─ DUQ_hourly.csv
|   └─ EKPC_hourly.csv
|   └─ FE_hourly.csv
|   └─ NI_hourly.csv
|   └─ PJME_hourly.csv
|   └─ PJMW_hourly.csv
|   └─ PJM_Load_hourly.csv
|   └─ est_hourly.parquet
|   └─ pjm_hourly_est.csv
| └─ index.html          | html code for the landing page
| └─ requirements.txt    | a file containing all dependencies needed to run the model code
└─ user_datetime_predictions.csv | file we import to the demo that contains predictive data

```

8.2 Mermaid Flowchart

Flowchart made using Mermaid.ai for visualization purposes.



8.3 GitHub Commit History

Here, we show details of update history of the project through pushes in GitHub

05/02, MacsenHigginsKSU : add home button to demo and make demo launch in same tab
 05/02, MacsenHigginsKSU : redesigned landing page in modern template
 04/30, MacsenHigginsKSU, Add link embeds and update KSU logo
 04/30, MacsenHigginsKSU, move web assets into folder and rename references
 04/30, MacsenHigginsKSU, remove keras file
 04/30, jacobsouthern-ksu, Merge branch 'main'
 04/30, jacobsouthern-ksu, Add html homepage, styles, and supporting images
 04/23, MacsenHigginsKSU, expanded comments for readability and ease of understanding
 04/23, MacsenHigginsKSU, add javascript to query CSV for prediction and show in box
 04/23, jacobsouthern-ksu, Merge branch 'main'
 04/23, jacobsouthern-ksu, minmax to datepicker
 04/23, MacsenHigginsKSU, Add code to create CSV of predictions for all user inputs
 04/02, jacobsouthern-ksu, Merge branch 'main'
 04/02, MacsenHigginsKSU, save model as keras file
 04/02, jacobsouthern-ksu, update html & add plotly html
 04/02, MacsenHigginsKSU, add graph for error over time
 04/02, jacobsouthern-ksu, add plotly import in requirements.txt
 04/02, MacsenHigginsKSU, Merge pull request #1
 03/26, MacsenHigginsKSU, add day_of_week feature
 03/26, MacsenHigginsKSU, initial SplineTransformer implementation
 03/26, jacobsouthern-ksu, add index.html
 03/26, MacsenHigginsKSU, negligible spacing/semantics changes
 03/19, MacsenHigginsKSU, add requirements doc
 03/03, MacsenHigginsKSU, visualize truncated actual vs. predicted values

03/03, MacsenHigginsKSU, add gitignore, embed csv reference
 02/28, macsenhiggs, initial commit

9.0 Conclusion

In totality, we are satisfied with the results we were able to produce and the process we used in achieving them. With results averaging an R^2 value around 0.65 between predicted and actual usage values, the accuracy of our model is decent, although results still have room for improvement. Our model is effective at understanding the rise and fall of electricity usage within the course of a day but still struggles to predict and respond to anomalous behavior over the course of multiple days, such as a possible heat wave causing a spike in usage. These shortcomings could be attributed to the simplicity of our input data, as the date and time alone would be generally insufficient information to predict the usage of a power grid. If we had data for mean temperature at each timestamp, for example, we would expect improved accuracy. In addition, the LSTM structure of our model itself may put a cap on its overall accuracy. If we were to continue this project into the future, we would continue to tweak the internal structure of the model to aid in its training. In addition, we believe the model would work best in real-time, constantly re-training based on recent usage timestamps so that it can anticipate spikes in usage before they come, allowing the electricity provider to prepare for anomalous fluctuations.

10.0 Appendix

10.1 Data Sources

Our data was obtained from the following Kaggle Dataset:
<https://www.kaggle.com/datasets/robikscube/hourly-energy-consumption>, which falls under public domain via the CC0 creative commons license.

10.2 Definitions

The following acronyms and terms have been used throughout this document:

Term/Acronym	Definition
LSTM	Long-Short-Term Memory — A specific structure of neural network.
PJM Dataset	Our public dataset of date/time and energy usage from Kaggle.
GitHub Pages	A free static web-hosting service provided by GitHub
TensorFlow	An open-source machine learning library for Python
R^2	Coefficient of determination — the proportion of the variation in the dependent variable that is predictable from the independent variable(s).

10.3 Points of Contact

The following people can be contacted regarding this document:

Primary Contact	
Name	Macsen Higgins
Title/Organisation	Student, Kennesaw State University
Phone	404-480-3123
Email	mhiggi36@students.kennesaw.edu
Secondary Contact	
Name	Jacob Southern
Title/Organisation	Student, Kennesaw State University
Phone	865-898-8476
Email	jsouth28@students.kennesaw.edu